

AAI-540 ML Design Document Template

Team Info

Project Team Group #: 3

Authors:

Luis Lopez

Andrew Roehr

Aaron Ramirez

Business Name: Maintain your vehicle with AI

Publication Date: 6/23/2024

Team Workflows

GitHub Project Link: [Link](#)

Asana Board Link: [Link](#)

Team Tracker Link: [Link](#)

Project Scope

Project Background:

Anyone who owns a vehicle knows one thing for certain: it will eventually require maintenance. Whether the vehicle is powered by electricity, gasoline, or diesel, some form of service is inevitable. The goal of this model is to classify whether a vehicle will require maintenance or not(binary), although it does not specify the exact type of service needed.

In a business context, especially when managing a fleet of vehicles, preventing on-the-road breakdowns is critical. This makes it essential to capture the most relevant features, such as maintenance history, current mileage, and engine type. By leveraging classification techniques, we can proactively address potential issues, reducing operational costs and minimizing unexpected failures.

Problem Statement:

Vehicle maintenance is a critical aspect of fleet and personal vehicle management. Unexpected breakdowns can lead to high repair costs, downtime, and safety risks. Traditional approaches rely on reactive service models or static mileage-based schedules, which may not reflect actual wear or use conditions. Our goal is to develop a predictive machine learning

system that classifies whether a vehicle will need maintenance, enabling proactive servicing and reducing the chance of failure.

The challenge lies in learning meaningful patterns from historical maintenance data, mileage, engine characteristics, and service history, especially in the presence of imbalanced class distributions. The system must deliver reliable predictions while operating within a scalable, secure infrastructure that supports continuous integration and monitoring.

Impact Measurement:

The primary way we will measure impact is through model evaluation metrics. Specifically F1-score, precision, and recall, to ensure the system balances false positives and false negatives. These metrics are critical in maintenance prediction, where both missed detections and false alarms incur costs.

From a business standpoint, model success will be reflected in reduced emergency maintenance events, improved service planning, and optimized vehicle uptime. By tracking real-world outcomes after integration (e.g., % reduction in unexpected breakdowns), we can demonstrate the model's tangible value in supporting proactive vehicle care.

Technical Background:

When working with binary classification models, it is common to evaluate performance using metrics such as accuracy, precision, recall, and the F1-score. This project follows the same approach, with the objective of minimizing the binary cross-entropy loss function while maximizing overall model accuracy.

The dataset used in this project is a synthetic dataset sourced from Kaggle, containing 50,000 observations and 20 features, including the target variable. As the dataset is highly imbalanced, it will be necessary to apply resampling techniques, such as oversampling the minority class or undersampling the majority class, to ensure the model does not become biased.

Prior to modeling, we will address data irregularities, including missing values, outliers, and inconsistent data types. A thorough sanity check and exploratory data analysis (EDA) will follow, allowing us to uncover meaningful patterns and relationships within the data. These insights will guide feature engineering efforts aimed at enhancing the model's predictive power.

Key features expected to influence model performance include `maintenance_history`, `miles`, `vehicle_age`, `engine_size`, and `service_history`. For model selection, we will consider both logistic regression for its simplicity and interpretability and a shallow neural network with a single

output neuron and a sigmoid activation function, leveraging the dataset's structure and feature complexity for improved classification performance.

Goals vs Non-Goals:

Goals

- Achieve high performance using precision, recall, and F1-score, not just accuracy
- Minimize loss error
- Handle data imbalance through resampling methods
- Engineer meaningful features from raw data
- Compare performance of logistic regression vs. shallow neural network
- Build an end-to-end MLOps system architecture

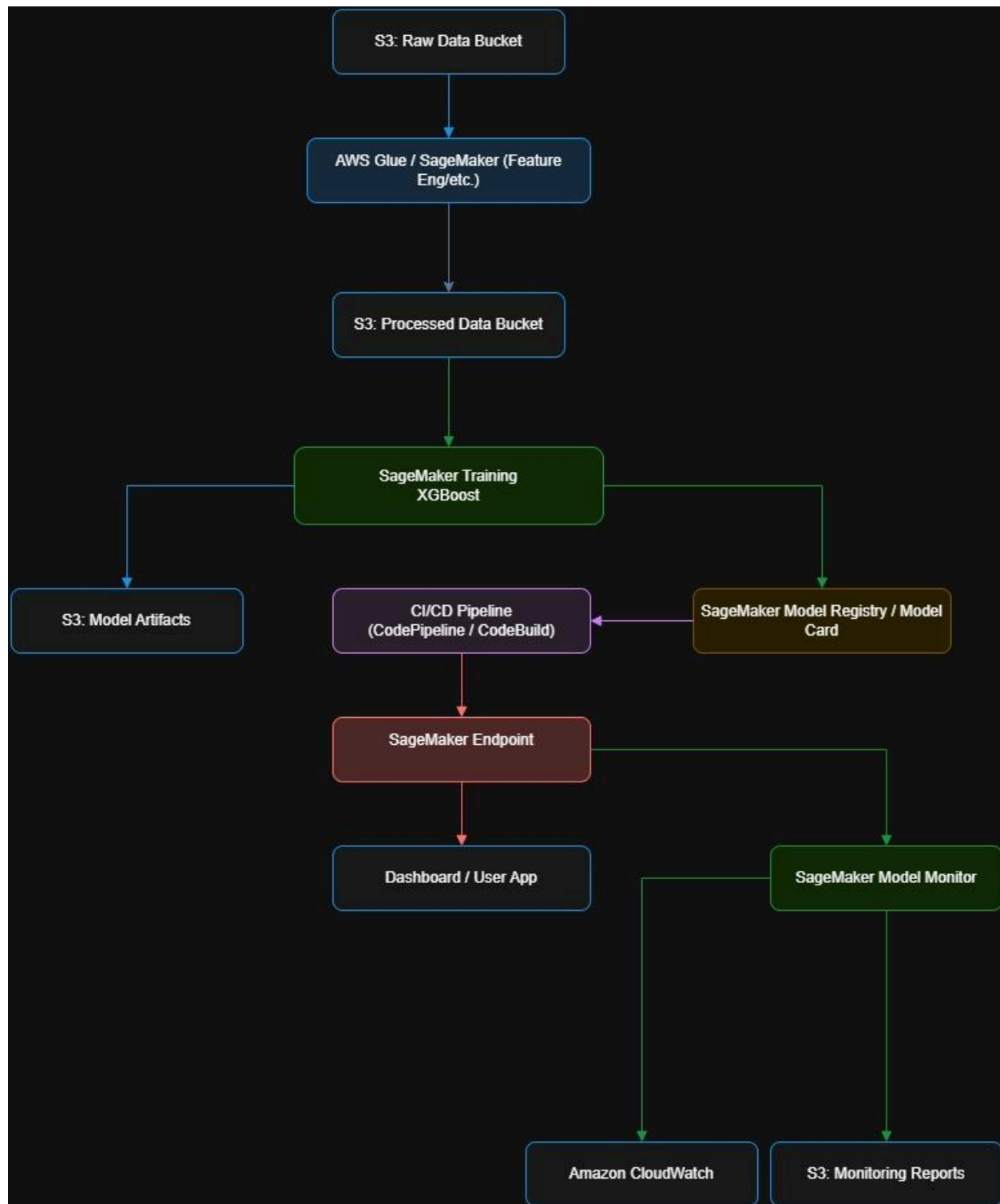
Non-Goals

- Using deep neural networks
- Creating a user-facing application or UI for model input/output
- Conducting in-depth business cost/ROI analysis from model predictions

Solution Overview

This AWS-based machine learning system streamlines predictive vehicle maintenance by securely storing raw data in S3, transforming and engineering features with AWS Glue and training both linear regression and shallow neural network models in Amazon SageMaker. Model artifacts are managed in S3, while model versions and metadata are tracked in the SageMaker Model Registry and documented with Model Cards. Automated deployment is handled through a CI/CD pipeline, with endpoints exposed for real-time or batch predictions to business applications and dashboards. The system includes comprehensive monitoring via SageMaker Model Monitor and CloudWatch, ensuring data quality, model performance, and infrastructure health, all designed to support proactive maintenance, business efficiency, and customer value.

System Architecture Diagram: [System Diagram](#)



Data Sources:

The data source is from [Kaggle](#), it is a synthetic collection of over 50,000 vehicles and their maintenance data. Some features it includes is model, mileage, maintenance, age and more. This was chosen as this dataset mimics real world data and can be used to help show a model for predicting maintenance for vehicles.

The risk will be the balance of the data. It will need to have oversampling of the underrepresented features and vice versa for the over represented features. The data is synthetic so there are no sensitive data or incomplete records.

Data Engineering:

The data will be stored within AWS services. We will be utilizing S3 for its scalability, and ability to use all the other services seamlessly, like Athena, Glue and SageMaker. The data will need to be preprocessed to where we will be cleaning the data, performing feature engineering, one hot encoding and doing EDA to see what is best.

Training Data:

The plan is to do an 80/10/10 split, 80% training, 10% validation, 10% test. No additional labeling is required since it will be a binary classification of if a vehicle needs maintenance or not.

Feature Engineering:

Initially we plan to include all features. This changed where we grouped service history, accident history and reported issues. We also grouped tire condition, brake condition and battery status as well and created the miles per year. Finally we created total hist service from combining hist record and service status and created the last service date and week.

We can do categorical encoding for model, fuel, transmission and owner type. Finally, we can do normalization and scaling to make all data points be similar and continuous.

Model Training & Evaluation:

We will train the model on SageMaker which will allow us to create a model store to store all the attempts we have. We will have linear regression and a shallow neural network. Linear

regression will not have hyper parameter but the neural network will have hidden layers, number of neurons, activation function, learning rate, batch size and epochs.

We will evaluate based off Accuracy, precision, recall and F1 score. We can also look at the ROC-AUC for more information on the performance.

Model Deployment:

We will deploy on an endpoint through SageMaker. We can scale the instance size as needed starting at an ml.t2.medium and going up to as high as a ml.m5.large if needed. We can have the prediction be on a schedule for like a week report for a business for the user vehicle. The results do not need to be immediate to either of the business or the owner for the vehicle.

Model Monitoring:

Model Monitoring is a critical component of maintaining reliable and high-performing machine learning systems in production. To monitor the model itself, we will implement automated performance tracking using AWS SageMaker Model Monitor and CloudWatch. This setup will continually measure key metrics such as accuracy, recall, F2 score, and latency, allowing us to detect any performance drift or degradation. Alerts will be configured to notify the team if these metrics fall below established baseline thresholds, ensuring prompt intervention.

For infrastructure monitoring, we will leverage CloudWatch to track essential resource metrics including CPU and memory utilization, network and error rates across all deployed endpoints. This will help us proactively identify bottlenecks or resource exhaustion issues, enabling timely scaling or remediation to maintain service availability and cost efficiency.

To monitor data, we will enable data quality monitoring features within SageMaker, which continuously inspect incoming data for anomalies, data drift, and distribution changes compared to the baseline. This approach ensures that the data fed to the model remains consistent with the original training data, reducing the risk of unseen or corrupted inputs negatively impacting model predictions. Combined, these monitoring strategies provide comprehensive oversight of the entire machine learning workflow, supporting robust and resilient operations.

Model CI/CD:

The checkpoints it will contain will be source control integration, data validation and model evaluation. These 3 will make sure the code is good, the data is good and the model is working well with the data it is receiving. There will also be rollback capability if deployments fails. For

test we will do unit test, data validation test, and model performance test. This will make sure the individual data is still correct, can check if we are missing data or have outliers and finally make sure our models are improving compared to each other.

Security Checklist, Privacy and Other Risks:

Will this store or process Personal Health Information (PHI)?

Will this store or process Personal Identifiable Information (PII)?

Will user behavior be tracked and stored?

Will this store or process credit card information?

If you answered yes to any of the above questions, please justify.

What S3 buckets will this application read from or write to?

What data bias should be considered?

Will your model have potential for bias along sensitive features (race, ethnicity, gender, age, religion, disability, sexual orientation, or other personal attributes)

Are there any ethical concerns with the data or business problems that should be addressed?

No PHI, PII, or credit card information will be stored. The most of user behavior will be stored is when the vehicle was last serviced, but this can't be tracked to any person from this one data point alone or this connected to any data points we have.

We will have an S3 bucket for raw data, processed data, model artifacts and vehicle maintenance monitoring. Data bias will be considered for over and under representation of columns. We will not have any bias for race, ethnicity, gender, age, religion, disability, sexual orientation, or other personal attributes. One ethical consideration is diagnosing a vehicle for maintenance based on their insurance premium. Monitoring this feature can be helpful to avoid any misclassification due to the insurance premium value.

Future Enhancements:

Provide at least 3 ways you would improve your ML system if you had more time or additional resources.

Cost Optimization and Warranty Analytics:

Leverage the model's predictions to optimize maintenance plans for cost savings, and analyze warranty claims data to identify patterns of early failure or potential fraud. This can help the business reduce operational expenses and manage warranty risk more effectively.

Integration with Parts Inventory and Supply Chain:

Connect the ML system with inventory management and supply chain platforms to forecast parts demand based on predicted maintenance needs, reducing overstock and avoiding out-of-stock delays.

Automated Data Feedback Loop:

Implement a feedback collection mechanism where new maintenance records or user-reported issues are automatically logged and incorporated into the training data pipeline. This enables continuous model retraining and adaptation to new patterns.